

CC91 校园论坛系统 — 项目总结报告

项目周期：2026 年 4 月 – 2026 年 6 月

团队规模：11 人

代码提交：156 commits (当前分支)

版本：v2.0 (微服务架构)

一、项目概述

CC91 是一个面向校园场景的现代化论坛系统，采用 Spring Boot 微服务架构 + React 19 前端，支持用户认证、帖子管理、评论互动、通知推送、管理后台等完整功能闭环。

经过多轮迭代，项目从单体架构演进为 **7 个微服务 + API 网关 + 服务注册中心** 的分布式系统，并通过 Docker Compose 实现一键部署。

1.1 架构演进时间线

时间	阶段	关键产出
2026-04	单体基线 (M1/M2)	Spring Boot 单体后端 + React 19 前端，用户/资料模块完成
2026-04-13	AI 协作体系建立	Lead/Developer/QA 三类智能体协议 (CLAUDE.md / developer.md / qa.md)，4 个 Skills 落地
2026-05	测试补齐	后端 JUnit 5 + MockMvc 单元/集成测试体系，前端 Vitest + RTL 组件测试
2026-05-28	CodeGraph MCP 接入	引入代码知识图谱，加速符号搜索与调用链追踪
2026-06-09 ~ 06-10	微服务拆分	单体 → Eureka + Gateway + 6 业务微服务，OpenFeign + Resilience4j 通信
2026-06-11 ~ 06-12	容器化	Docker Compose 编排 9 个容器，多阶段构建，healthcheck 启动顺序
2026-06-13 ~ 06-14	监控与压测	Prometheus + Grafana + 5 条告警规则，万级数据下 8 场景压测

二、已实现的功能

2.1 核心功能矩阵

模块	功能	状态
用户系统	注册 / 登录 / 退出 / 邮箱验证 / 密码重置	✓
	JWT 双 Token 机制 (Access + Refresh)	✓

	账户锁定（多次失败触发，阈值由后端配置控制）	✓
	个人资料编辑、头像上传	✓
论坛核心	版块分类浏览	✓
	发帖（支持草稿/发布双状态）	✓
	帖子编辑、删除（权限控制）	✓
	评论 + 多级嵌套回复	✓
	点赞 / 收藏（toggle 机制）	✓
	全文搜索	✓
通知系统	评论/回复实时通知	✓
	通知已读/全部已读	✓
	未读数角标	✓
管理后台	用户管理（封禁/解封）	✓
	分类管理（增删改排序）	✓
	公告管理（发布/置顶）	✓
	内容审核（举报处理）	✓
安全性	bcrypt 密码哈希	✓
	XSS 双层防护（DOMPurify + HtmlSanitizer）	✓
	RBAC 权限控制	✓
	JWT 无状态认证	✓
	文件上传多层校验	✓

2.2 页面与路由

路径	页面	认证
/	首页（公告 + 版块 + 最新帖子 + 热门）	否
/login	登录	否
/register	注册	否
/category/:id	版块帖子列表	否
/posts/:id	帖子详情 + 评论	否
/search	搜索结果	否

/profile/:username	个人主页	否
/profile/edit	编辑资料	是
/notifications	通知列表	是
/admin/*	管理后台	管理员
/announcements/:id	公告详情	否

三、掌握的关键技术

3.1 技术栈总览

层级	技术选型	说明
后端框架	Spring Boot 3.2.12	Java 17, Maven
微服务治理	Spring Cloud 2023.0.0	Eureka + Gateway + OpenFeign
安全框架	Spring Security + JWT (jjwt 0.12.3)	无状态认证 + RBAC
数据库	MySQL 8.0.46	Flyway 9.x 版本化管理
前端框架	React 19 + Vite + TypeScript	函数组件 + Hooks
状态管理	@tanstack/react-query	服务端状态缓存
断路器	Resilience4j 2.1.0	熔断、降级、重试
监控	Spring Boot Actuator + Micrometer + Prometheus + Grafana	全链路指标采集
容器化	Docker + Docker Compose	9 个容器一键部署
测试	JUnit 5 + MockMvc + Vitest + RTL	分层测试
前端安全	DOMPurify	XSS 前端层防护
后端安全	HtmlSanitizer + BCrypt	XSS 后端层防护 + 密码哈希

3.2 智能技术应用 (Claude Code + MCP Server + Skills)

本项目在开发过程中深度应用了 AI 辅助开发技术，形成 "Agent + MCP + Rules + Skills" 的完整协作闭环。

3.2.1 Claude Code 作为开发平台

- 角色：Claude Code 担任项目 Lead，通过 CLAUDE.md 定义完整的工作协议（团队、5 阶段 workflow、审查清单、技术栈声明）
- Subagent 体系：
 - Lead（项目根 CLAUDE.md）：需求理解、任务拆解、代码审查、最终验收

- **Developer Agent** (`.claude/agents/developer.md`) : 全栈开发, 配备 5 个专业技能
- **QA Agent** (`.claude/agents/qa.md`) : 测试驱动, 强制运行测试并输出报告
- **workflow**: Phase 1 理解规划 → Phase 2 开发 → Phase 3 测试 → Phase 4 审查 → Phase 5 收尾

3.2.2 MCP Server (Model Context Protocol)

MCP 是 Anthropic 提出的开放协议, 为 AI 智能体提供连接外部工具和数据源的标准接口。本项目配置的 MCP Server:

MCP Server	用途	实际应用
CodeGraph	基于 Tree-sitter 的代码知识图谱, 提供亚毫秒级符号搜索	代码导航 (codegraph_context 一键获取任务上下文)、调用追踪 (codegraph_trace 跟踪完整调用链)、影响分析 (codegraph_impact 评估变更影响范围)。项目共索引 308 个文件, 4284 个符号, 6998 条边, 替代传统 grep/read 循环
Playwright	浏览器自动化	QA 智能体的 E2E 测试与页面视觉验证, 动态决策"点击哪个按钮、检查哪个元素"
IDE (executeCode / getDiagnostics)	Jupyter Kernel 执行 + VS Code 诊断	在 Jupyter 中执行 Python 压测脚本, 获取语言服务器诊断信息

选用依据: Bash 可以运行预编写脚本, 但无法在运行时动态决策。CodeGraph 让 AI 像 IDE 一样理解代码结构 (函数→调用者→被调用者→影响范围), Playwright 让 QA 像真人一样"看着页面操作"。文件操作、构建测试、Git 管理等日常任务 Claude Code 内置工具已完全胜任, 避免过度工程化。

3.2.3 规则系统 (Rules)

规则系统约束 AI 的输出边界和行为规范:

规则文件	约束范围
CLAUDE.md (项目根目录)	Lead 智能体的工作协议: 团队定义、5 阶段 workflow、审查清单、技术栈声明
.claude/agents/developer.md	Developer 智能体行为约束: 技术栈、核心职责、工作原则、项目结构约定
.claude/agents/qa.md	QA 智能体行为约束: 测试质量红线 (禁止测试框架行为、必须测试业务分支)、测试优先级、报告格式

选用依据: 将团队经验固化为可执行约束。例如 QA 规则明确禁止"测试框架行为" (如 @Valid 注解、Bean 注入), 强制要求"测试业务逻辑分支" (如登录失败计数、账户锁定触发), 直接提升测试有效性。

3.2.4 技能 (Skills) 体系

技能模块将领域最佳实践沉淀为可复用的知识单元。项目配置了 5 个 Skills:

Skill	来源	用途
-------	----	----

java-springboot	社区开源	Spring Boot 最佳实践：项目结构、依赖注入、Web 层、数据层、日志、测试、安全
springboot-patterns	社区开源 (ECC)	Spring Boot 架构模式：REST API 设计、分层服务、DTO、缓存、异步、限流、错误处理、可观测性
typescript-react-reviewer	社区开源	TypeScript + React 19 代码审查：反模式检测、状态管理、React 19 Hook 陷阱、类型安全
vercel-react-best-practices	Vercel 官方	React 性能优化 60+ 条规则：异步瀑布、Bundle 优化、服务端性能、重渲染、渲染策略
web-design-guidelines	社区开源	Web 界面设计规范

选用依据：技能的可组合性使得"一个 Developer 智能体 + 多个 Skills"比"多个 Developer 智能体"更具经济性和可维护性。技能通过 `skills-lock.json` 版本化锁定，保证旧流程可回溯。

四、开发流程与团队协作

4.1 开发流程

需求分析 → 任务拆分 → Developer 实现 → QA 测试 → Lead 审查 → Git 提交

更详细的 5 阶段工作流：

阶段	Lead	Developer	QA
Phase 1 理解规划	与用户对话、TaskCreate 编排依赖	—	—
Phase 2 开发	SendMessage 分配任务 (含验收标准)	阅读现有代码、参考 Skills、实现代码、自测	—
Phase 3 测试	SendMessage 分配测试任务	—	运行 <code>mvn test / npm test</code> 、按 <code>qa.md</code> 报告格式输出
Phase 4 审查	按审查清单检查 (安全/API/错误/质量)	修复阻断问题	补充回归
Phase 5 收尾	确认任务完成、提交变更	—	—

4.2 团队角色分工 (11 人)

成员	软件工程角色	主要职责	负责维护的智能体配置
----	--------	------	------------

徐哲楷	项目负责人 / 产品统筹	控制范围、确认里程碑、统一验收口径	Lead 智能体 (CLAUDE.md 协议、审查清单)
蔡致	前端工程师	页面结构、路由、表单、交互实现	Developer + typescript-react-reviewer
刘一鸣	前端工程师	状态管理 (React Query)、接口联调、错误处理	Developer + vercel-react-best-practices
项方灿	前端工程师	样式系统、响应式适配、可用性优化	Developer + vercel-react-best-practices
王建皓	后端工程师	登录、注册、鉴权、会话管理 (Spring Security + JWT)	Developer + springboot-patterns
李明睿	后端工程师	帖子、评论、内容治理接口	Developer + springboot-patterns
罗新鹏	后端工程师	数据模型 (JPA Entity + Flyway 迁移)、校验与异常处理	Developer + java-springboot
朱城弘	测试工程师	测试用例设计、后端接口测试 (JUnit 5 + MockMvc)	QA + qa.md 规则维护
陈瑜凡	测试工程师	前端组件测试 (Vitest + RTL)、回归验证	QA + qa.md 规则维护
陈元煦	DevOps 工程师	环境配置、构建发布、数据库运维、Docker 容器化	自动化脚本 (Maven/npm/Docker)
王帆	安全与审查工程师	安全基线审查、代码审查、知识沉淀	CLAUDE.md 审查清单 + 安全规则

4.3 协作机制

- **任务驱动**: 每个任务包含唯一 ID、背景目标、实现范围、验收标准、依赖关系
- **阻塞升级**: 任务阻塞超 30 分钟自动上报, 超 2 次未解决则由 Lead 调整策略
- **问题跟踪**: docs/issues.md 实时维护未修复 Bug, P0/P1/P2 优先级管理
- **代码审查**: 每次提交前通过安全检查、API 设计、错误处理、代码质量、测试质量五大维度审查

4.4 Git workflow

- **分支策略:** `main` → `develop` → `feat/*` / `fix/*`
 - **提交规范:** Conventional Commits (`feat:` / `fix:` / `refactor:` / `test:` / `docs:` / `chore:`)
 - **PR 流程:** 功能分支 → `develop`, 通过审查后合并
 - **AI 产物约束:** AI 生成代码必须绑定到具体任务编号, 不允许直接合并, 必须经过人工确认
-

五、个人心得体会

5.1 徐哲楷 — 项目负责人 / Lead 协议维护

- **承担角色:** 项目负责人 (统筹) + Lead 智能体协议 (CLAUDE.md) 维护
- **主要挑战:**
 1. **AI 智能体协作体系从零搭建** — 需要设计一套可执行的 `Lead` → `Developer` → `QA` 多智能体工作流, 让 AI 能像真实团队一样分工协作, 而不是独立工作。初期尝试过一个 Agent 独立开发, 结果代码质量参差不齐。
 2. **架构拆分期间保持可运行** — 从单体到微服务的架构迁移涉及 277 个文件变更、6 次大规模重构, 需要在“持续交付”和“架构改造”之间找到平衡, 不能让项目长期处于不可运行状态。
 3. **11 人并行开发的冲突管理** — 前端 3 人、后端 3 人、测试 2 人、DevOps 1 人、安全 1 人, 每人依赖不同的 AI Agent 配置, 偶尔会出现 Git 冲突和接口契约不一致。
 4. **AI 生成代码的质量一致性** — 不同成员使用的 AI Agent 版本和技能配置有所不同, 生成的代码风格和架构一致性难以保证。
- **解决方法:**
 1. **三智能体协议 (CLAUDE.md + developer.md + qa.md)** : 将 Lead 定位为“项目经理 + 架构师”, Developer 只负责实现 (配备专业技能与 MCP), QA 只负责测试 (强制运行测试)。通过 `SendMessage` 机制实现 `Lead` → Agent 的任务分派, 每个任务附带完整的验收标准模板 (功能验收 + 测试验收 + 边界验收 + 回归范围), 确保 Developer 产出可被 QA 验证。
 2. **5 阶段工作流 (Phase 1-5)** : 定义“理解规划 → 开发 → 测试 → 审查 → 收尾”的刚性流程, 每个阶段有明确的进入条件和完成标准。架构拆分期间采用“小步多次”策略——每次只拆 1-2 个服务, 验证通过后再拆下一个, 避免大规模重构带来的风险。
 3. **任务驱动 + 阻塞升级机制**: 每个任务有唯一 ID、背景目标、依赖关系、阻塞记录 (`blocker` / `blocked-by` / `next-action` / `escalation-owner`)。阻塞超 30 分钟自动上报, 超 2 次由 Lead 重新拆分任务或变更路径。Git 提交采用 Conventional Commits 规范, 按逻辑分组提交, 减少冲突范围。
 4. **审查清单 + issues.md 问题跟踪**: 每次提交前通过安全性、API 设计、错误处理、代码质量、测试质量五大维度审查。发现的问题立即写入 `docs/issues.md` (P0/P1/P2 分级), 修复后删除——保持清单简洁, 只反映未完成工作。
- **收获与反思:**
 1. **在 AI 协作开发中, 协议比代码更重要。** 一开始我低估了 CLAUDE.md 的作用, 只是把它当成软件工程中的 SRS 一样的文档。但实际跑起来后才发现, AI 需要精确写明的开发流程协议。不然 Developer 会直接跳过测试, QA 也会在不完整上下文下修改代码。后来我把流程拆成 Phase 1-5, 并且把验收标准、阻塞机制全部显式化之后, 行为才稳定下来。这件事让我感觉: 协议越精确, AI 开发越精确, 生成的代码更严谨优美。
 2. **写给 AI 的 Prompt 也很重要, 需要全面严谨。** 最开始我写 Prompt 时很随意, 就是口头的几句话。结果就是需要来回返工, 后来我将 Prompt 改成结构化任务后, 效果要稳定很多。提示词工程也是 AI 协作开发的重要领域之一, 需要不断探索学习。

3. **AI 代码幻觉需要有流程兜底。** AI 会编造不存在的 API、假设框架行为、不问约束就做架构决策。需要靠“审查清单机制”兜底——需要在合并前由 Lead 逐项检查安全/API/错误处理/代码质量/测试质量 5 个维度。
4. **AI 协作开发效率大大提高，但也产生了新瓶颈。** AI 间通信零延迟，产出速度很快，但远超出了人类审查能力。需要通过**分层审查**的方式加快工作效率：AI 自测 → QA Agent 集成测试 → Lead 架构审查。
5. **总的反思与复盘：** 这次项目让我参与了一次多人多 Agent 协作开发的实践，是我之前没有经历过的经历，在这次的实践中，我收获了对多智能体开发的经验。AI 有很强的代码产出能力，可以快速产生大量代码构建项目，降低了代码编写成本，提高了项目完成效率。但它产出的稳定性和质量依赖文档、验收标准、流程的设计，把问题转移到了任务定义、过程管理和结果验证上。多智能体的开发合作也需要严谨的文档、流程设计来实现，如何设计一个让 AI 稳定工作的协作系统也许是未来的重点方向。

5.2 蔡致 — 前端工程师（页面结构、路由、表单）

- **承担角色：** 前端工程师，负责页面结构、路由设计、表单交互；维护 Developer 智能体 + typescript-react-reviewer 技能
- **主要挑战：** 难以让 Agent 理解期望得到的美术风格
- **解决方法：** 采用 截图 + playwright MCP 为 AI 提供充足而精确的原始信息。
- **收获与反思：** 在让 AI 进行创作时，文字描述会丢失许多细节信息，在上下文充足的情况下，可以考虑将原始信息（图片、网页 DOM 等）作为输入，从而让 AI 的产出风格收敛到期望的结果。

5.3 刘一鸣 — 前端工程师（React Query、接口联调）

- **承担角色：** 前端工程师，负责状态管理（React Query）、接口联调、错误处理；维护 Developer + vercel-react-best-practices
- **主要挑战与解决方法：**
 - **试验的核心收获：状态管理即缓存协议：** CC91 论坛涉及 20+ 页面，数据全部来自服务端。我的直接体会是：前端没有自己的状态，只有服务端状态的缓存。这和传统的前端状态管理（如 Redux）的思维模式完全不同——不再关心数据怎么存，而关心数据怎么和服务器保持同步。React Query 的价值不在于发请求，而在于提供了可预期的缓存失效协议。通过分层 queryKeys 设计（帖子列表与详情共享 ['posts'] 前缀），一次 invalidate 驱动所有相关视图刷新，而手动维护这套一致性几乎是不可能的。在大规模系统中，数据一致性不是靠“每次都请求最新”实现的，而是靠明确的失效规则保证最终一致。这里的关键转变是从“拉”到“推”的思维——不主动获取，而是声明依赖和失效条件。
 - ****联调的隐性挑战：异常是常态：** **token 自动刷新是联调中最花精力的模块。核心逻辑简单，但边界条件多：多个请求同时 401 如何只刷新一次？刷新请求也 401 怎么办？多标签页同时刷新如何避免竞态？这些问题在单页面开发时几乎不会被注意到，但在真实环境中一定会出现。在试验中我学到的是：在大规模信息系统里，异常路径不是边界情况，而是默认运行环境。页面可以因为后端某个服务抖动而局部降级，但不能全军覆没。API 层按后端领域（帖子、评论、用户）而非前端页面拆分——页面布局频繁变，业务领域边界不会变。这一经验在后续对接微服务架构时尤为关键，API 层的领域划分直接对应了后端的服务边界。
 - **错误处理即故障隔离：** 每个数据组件处理 loading、error、empty 三态，ErrorBoundary 兜底渲染异常。这不是 UI 层面的“锦上添花”，而是系统可靠性的最后一环。设计思路与微服务架构同源：一个页面崩溃不该波及其他页面。ErrorMessage 组件用局部状态而非全局消息总线，也是同一逻辑——**能局部化的状态就不要全局共享**。在多个微服务协作的场景下，任何一个服务的不可用都应该被限制在局部体验中，而不是扩散为全局故障。

- **收获与反思：** 试验中形成了几条个人体会，它们共同指向一个结论：**大规模系统开发的关键不是解决复杂问题，而是让简单问题不叠加成复杂问题。**

1. **状态管理的核心是失效规则**——系统膨胀后手动同步必然出问题，需要规范的缓存协议来约束
2. **异常是默认路径**——系统可靠性取决于异常处理能力，而非理想路径上的表现
3. **模块边界由业务领域定义**——按页面拆分短期方便，按领域拆分才能随规模保持稳定
4. **AI 辅助的效率取决于指令规范程度**——Agent 技能模块的有效性建立在约定明确的前提之上，人和工具在清晰的框架内协作效率最高

5.4 项方灿 — 前端工程师（样式、响应式、可用性）

- **承担角色：** 前端工程师，负责样式系统、响应式适配、可用性优化；维护 Developer + vercel-react-best-practices
- **主要挑战：** 可访问性的落地。键盘导航、屏幕阅读器支持、焦点可见性等无障碍需求在初期容易被忽视
- **解决方法：** 进行可用性细节打磨。添加了键盘跳过导航链接、空状态友好提示、防抖搜索。移动端设计了右侧滑出式导航面板，兼顾功能完整性和触屏操作体验。
- **收获与反思：** AI 往往倾向于输出功能正确但边界状态考虑不足的代码——例如只写"数据正常展示"的分支，不写 loading/empty/error。通过在任务描述中进行明确约束，可以显著提升 AI 输出质量。

5.5 王建皓 — 后端工程师（鉴权、会话）

- **承担角色：** 后端工程师，负责登录、注册、鉴权、会话管理（Spring Security + JWT）；维护 Developer + springboot-patterns
- **主要挑战：** 登录失败次数被事务回滚导致爆破防护失效；Refresh Token 轮转存在中间窗口期；邮件发送耦合注册流程造成事务回滚时误发。
- **解决方法：** `<font style="color:#000000;background-color:#FFFFFF;">@Transactional(noRollbackFor)</font>` 突破默认回滚策略；Token 先吊销再签发消除窗口期；`<font style="color:#000000;background-color:#FFFFFF;">@TransactionalEventListener(AFTER_COMMIT)</font>` 确保事务提交后才触发邮件。
- **收获与反思：** 深入理解了 Spring 事务传播机制与 Security 过滤器链，最大的体会是安全代码不能只考虑正常路径，异常路径下的行为往往才是真正的漏洞来源。

5.6 李明睿 — 后端工程师（帖子、评论、治理）

- **承担角色：** 后端工程师，负责帖子、评论、内容治理接口；维护 Developer + springboot-patterns
- **主要挑战：**
 - **任务描述模糊导致返工：** 第一轮说"实现帖子 CRUD"，Developer 异常处理用了 `RuntimeException`；第二轮明确指定"使用已有的 `ResourceNotFoundException` 和 `UnauthorizedException`"才一次通过。
 - **Agent 报告冗余：** Developer 和 QA 倾向复述自己做的所有事情（"创建了 X 文件"、"修改了 Y 方法"），占用大量上下文，而这些 Lead 直接看 git diff 就能知道。
 - **状态管理不透明：** 任务列表在团队切换时丢失；Agent 完成后只发消息，Lead 需主动验证是否真的完成；`TeamDelete` 在成员未退出时报错但错误信息不清晰。
 - **团队清理流程繁琐：** 发 `shutdown_request` → 等 `shutdown_response` → 调 `TeamDelete` → 手动清理残留目录，中间任何一步失败都需要手动介入。

- **审查与测试的顺序问题**：Developer 不会主动发现设计问题，QA 只测给定代码不会质疑设计；如果测试先跑，发现设计问题后又要重测。
- **解决方法**：
 - **QA Agent 的质量红线（“禁止”比“要求”更有效）**：在 qa.md 中明确写禁止事项——不测框架行为、不测 DTO getter/setter、不写无断言的测试。结果 52 个测试全部对应业务路径，零灌水测试。
 - **任务描述 = 技术约束 + 验收标准**：把 Agent 当成刚入职的聪明同事，把技术约束、参考文件路径、禁止事项都写清楚，不假设它“应该知道”。
 - **审查是唯一的质量兜底，且必须在测试之前**：Lead 实际读代码（不只看 Agent 摘要），第一轮审查发现异常处理问题修复后才进入测试，避免 QA 基于旧代码写测试。
 - **分批提交**：帖子管理 → 评论管理各一个独立 commit，可独立回滚；小步快跑，出了问题好定位。
 - **关键节点 Lead 亲自验证**：编译、测试 Lead 亲自运行命令验证，不完全依赖 Agent 的报告。
 - **发现的问题同步给全队**：让 QA 知道改了哪里，避免基于旧代码写测试。
- **收获与反思**：
 1. **告诉 Agent “不要做什么”比“要做什么”更有效**——Agent 天然倾向于写框架测试来凑数，禁止事项比要求事项更能约束行为。
 2. **把 Agent 当成刚入职的聪明同事**——不能假设它“应该知道”，技术约束、参考路径、禁止事项都要写清楚。
 3. **审查必须在测试之前，且要实际读代码**——Lead 的审查是唯一的质量把关环节，错序会导致 QA 重复工作。
 4. **信任但要验证**——Agent 说“测试通过了”，Lead 还是要自己跑一遍；关键节点亲自运行命令。
 5. **同一批 Agent 连续做类似任务时，规范 continuity 是效率杠杆**——第二轮（评论管理）因为第一轮（帖子管理）建立的规范已就位，几乎零往返，耗时从 ~30 分钟降到 ~15 分钟；如果中途重建团队，要重新告知规范。
- 6. **给定义 Agent 的人**：
 - Developer：要求“先读现有代码再动手”，指定参考模块，要求报告区分“做了什么”和“需要注意什么”。
 - QA：明确写禁止事项，要求先列出被测方法的所有分支再写测试，要求实际运行测试并附完整输出（不只是测试代码）。

5.7 罗新鹏 — 后端工程师（数据建模、Flyway）

- **承担角色**：后端工程师，负责数据模型（JPA Entity + Flyway 迁移）、校验与异常处理；维护 Developer + java-springboot
- **主要挑战**：协作比较麻烦，以及感受到技术选型、架构什么的对工作量影响挺大的
- **解决方法**：在制定的时候要多花些时间对话确认吧
- **收获与反思**：但感觉没有实际上手过干聊也聊不出个名堂。可能还是要项目经验

5.8 朱城弘 — 测试工程师（后端 JUnit 5 + MockMvc）

- **承担角色**：测试工程师，负责测试用例设计、后端接口测试（JUnit 5 + MockMvc）；维护 QA 智能体 + qa.md 规则
- **主要挑战与解决方法**：在本开发实验中，我主要负责后端测试工作。此次实践让我深刻认识到，在 AI Agent 深度参与的系统开发中，测试的边界与核心逻辑已发生实质性转变。面对 Agent 带来的动

态交互与非确定性推理路径，传统的接口验证与边界值测试已不足以覆盖潜在风险。后端测试的重心必须向工作流编排验证、工具调用结果评估以及多轮对话状态的一致性校验转移。这要求测试设计不仅要关注静态的数据一致性，更要建立针对Agent行为轨迹的动态评估体系，以保障复杂业务场景下系统逻辑的健壮性。

- **收获与反思：** 本次实验不仅验证了Agent在提升研发效率方面的可行性，也暴露了其在工程化落地时面临的质量挑战。这使我明确了在智能化开发范式下，测试工程师的核心价值在于构建适配AI特征的质量防线，确保系统在具备高扩展性的同时，维持严谨的工程纪律与可靠性。

5.9 陈瑜凡 — 测试工程师（前端 Vitest + RTL）

- **承担角色：** 测试工程师，负责前端组件测试（Vitest + RTL）、回归验证；维护 QA + qa.md
- **主要挑战与解决方法：** 作为本次项目的后端测试人员，在以项目合作者的身份参与到工作中时，我体会到“测试左移”的重要性。在开发初期提前介入接口设计与数据库表结构评审，帮助我们提前发现了不少潜在的逻辑缺陷，有效降低了后期修复的成本。在测试执行阶段，针对发帖、评论、权限校验等核心场景，我不仅设计了覆盖正常与异常流的测试用例，还引入了压力测试来模拟高并发场景，这暴露了数据库索引缺失和缓存策略不足的性能瓶颈。通过与开发同学的紧密协作，我们快速定位并解决了这些问题。
- **收获与反思：** 这次实验让我认识到，测试不仅是质量的“守门员”，更是产品交付的“助推器”。完备的自动化测试体系为项目的持续迭代构建了坚实的安全网，也让我在全球视野和风险把控能力上有了显著提升。总体而言我承担的责任不重，仰仗同组同学的辛勤付出，这个项目得以取得比较好的成果

5.10 陈元煦 — DevOps 工程师（容器化、构建发布）

- **承担角色：** DevOps 工程师，负责环境配置、构建发布、数据库运维、Docker 容器化；维护自动化脚本
- **主要挑战：** 没什么困难
- **解决方法：**
- **收获与反思：** ai开发比人类更有条理，好多初期分工的工作实际上在早期开发中已经完成了

5.11 王帆 — 安全与审查工程师（OWASP、XSS 防护）

- **承担角色：** 安全与审查工程师，负责安全基线审查、代码审查、知识沉淀；维护 CLAUDE.md 审查清单 + 安全规则
- **主要挑战：** 项目后期功能迭代频繁，需要在保证开发效率的同时兼顾代码安全性和审查规范的一致性。
- **解决方法：** 结合 OWASP Top 10 安全原则完善审查清单，对鉴权、XSS 防护、文件上传等关键模块进行重点检查，并持续更新安全规则。
- **收获与反思：** 加深了对 Web 安全风险和代码审查流程的理解，也认识到安全设计应尽早融入开发过程，而不是在项目后期集中补救。

六、技术维度深度分析

6.1 压力测试

6.1.1 测试方法

使用 Python 3.11 标准库（`concurrent.futures` + `urllib`）编写压测脚本，零第三方依赖。

6.1.2 测试配置

参数	值
并发线程	50
持续时间	15 秒 / 固定请求数 (500-1000)
目标服务	Gateway (port 9000)
数据规模	10,005 帖 + 90,014 评论 + 16 用户
测试脚本	docs/stress-test/stress_test.py
测试日期	2026-06-14

6.1.3 测试结果

场景	总请求	成功率	RPS	平均延迟 (ms)	P95(ms)	P99(ms)
只读基准 (categories + posts + announcements)	545	100%	34.86	1394.18	3175.86	4234.13
单接口 GET /api/categories	1009	100%	64.83	754.96	1253.21	1809.74
单接口 GET /api/posts	858	99.65%	28.56	935.90	1326.45	1836.19
单接口 GET /api/announcements	2175	100%	143.32	343.79	669.83	955.04
认证登录 POST /api/auth/login	623	100%	39.19	1236.72	2108.13	2386.25
读写混合 80/20	1000	100%	92.64	527.14	976.86	1811.74
并发写入 POST /api/posts	500	100%	119.92	402.43	742.50	1019.20
帖子详情 + 评论	1982	100%	129.21	380.65	574.37	757.19

6.1.4 瓶颈与优化方向

- **P0 瓶颈:** BCrypt 密码验证 (平均 1237ms) , 可通过登录限流 + Redis Session 缓存优化
- **P1 瓶颈:** 帖子列表查询 (GET /api/posts) , 万级数据下出现 3 次 30s 超时, 建议为 `posts(status, created_at)` 建立复合索引或引入分页缓存
- **优化方向:** 引入 Redis 缓存 (categories、announcements 等字典数据)、异步化浏览量更新、数据库连接池调优

6.2 微服务架构

6.2.1 拆分策略

采用 DDD (领域驱动设计) 的限界上下文方法, 按业务域拆分:

服务	端口	职责	数据表
Eureka Server	8761	服务注册与发现	—
API Gateway	9000	路由转发、负载均衡、重试	—
User Service	8081	认证、用户管理、密码重置	users, user_profiles, refresh_tokens, verification_codes
Forum Service	8082	帖子、评论、分类、点赞、收藏	posts, comments, categories, post_likes, bookmarks
Notification Service	8083	通知管理与推送	notifications
Content Service	8085	公告、举报、内容审核	announcements, reports
File Service	8086	文件上传与存储	文件系统

6.2.2 服务间通信

调用方	被调方	方式	用途
Forum	User	OpenFeign	获取帖子/评论作者信息
Forum	Notification	OpenFeign	评论时创建通知
Content	User	OpenFeign	公告作者信息
Content	Notification	OpenFeign	公告发布通知

6.2.3 收益分析

维度	单体	微服务
代码定位	98 个文件中搜索	10-20 个文件中定位
修改影响	全局回归	单服务测试
独立扩容	不可	按服务扩缩
故障隔离	单点全局不可用	降级后核心功能正常

6.2.4 过渡方案

当前采用**共享数据库模式**（所有服务共用 `cc91_db`，通过表名隔离），作为 DB-per-Service 的过渡形态。未来可逐步拆分为独立数据库，引入 Saga 模式处理分布式事务。

6.3 系统监控

6.3.1 监控架构

Grafana (可视化)
↑
Prometheus (采集 + 存储 + 告警)
↑ HTTP pull /actuator/prometheus
Gateway + 6 个微服务 + Eureka

6.3.2 工具选型

组件	选型	用途
指标暴露	Spring Boot Actuator + Micrometer	每个服务暴露 /actuator/prometheus
指标采集	Prometheus v2.48.0	每 15 秒拉取指标
可视化	Grafana 10.2.0	仪表盘展示
告警	Prometheus AlertManager	5 条告警规则

6.3.3 监控指标

类别	指标	用途
HTTP	http_server_requests_seconds_*	响应时间、吞吐、错误率
JVM	jvm_memory_used_bytes	堆内存使用
JVM	jvm_threads_live_threads	线程数
连接池	hikaricp_connections_active	数据库连接池
断路器	resilience4j_circuitbreaker_*	熔断状态

6.3.4 告警规则

告警	条件	级别
ServiceDown	up == 0 持续 1 分钟	Critical
HighErrorRate	5xx 错误率 > 10%	Warning
HighLatency	P95 > 2s	Warning
CircuitBreakerOpen	断路器打开	Warning
HighMemoryUsage	堆内存 > 85%	Warning

6.4 弹性设计

6.4.1 已落地的弹性措施

措施	实现方式	效果
----	------	----

断路器	Resilience4j CircuitBreaker	服务不可用时熔断，10s 后半开探测
重试机制	Gateway spring-retry + LoadBalancer retry	服务启动期间自动重试，避免 503
降级处理	Feign Fallback	通知发送失败时记录日志，不影响主流程
负载均衡	Spring Cloud LoadBalancer + Eureka	多实例时自动 Round Robin
连接池隔离	HikariCP 每服务独立连接池	单服务数据库压力不传染其他服务
容器重启	Docker restart: unless-stopped	服务异常退出自动恢复
健康检查	Spring Boot Actuator + Docker healthcheck	MySQL、Eureka 就绪后才启动下游

6.4.2 可引入的进阶弹性策略

策略	适用场景	推荐方案
舱壁隔离	防止某接口耗尽线程池	Resilience4j Bulkhead，为不同 API 分配独立线程池
限流	防止暴力破解登录	Bucket4j 令牌桶，登录接口 10 次/分钟
服务降级	论坛浏览不受通知服务影响	通知不可用时返回空列表，不阻塞帖子加载
异步解耦	通知、浏览量更新	RabbitMQ 消息队列，削峰填谷
分布式追踪	排查跨服务调用延迟	Micrometer Tracing + Zipkin

6.5 安全性保障

6.5.1 安全措施总览（按 OWASP Top 10）

OWASP	类别	措施	状态
A01	权限控制失效	RBAC 角色控制 + @PreAuthorize + InternalApiAuthFilter	✓
A02	加密机制失败	BCrypt 密码哈希，JWT HMAC-SHA 签名	✓
A03	注入攻击	JPA 参数化查询，DOMPurify + HtmlSanitizer 双重 XSS 防护	✓
A04	不安全设计	无状态 JWT，DTO 与 Entity 分离	✓
A05	安全配置错误	环境变量管理密钥，无硬编码，CORS 白名单	✓
A06	过时组件	mvn dependency-check:check + npm audit	✓

A07	认证失败	双 Token 机制，账户锁定（多次失败触发），Refresh Token 撤销	✓
A08	数据完整性	Flyway 版本化迁移，数据库 schema 可追溯	✓
A09	日志失败	异常全量记录，登录失败 WARN 日志	✓
A10	SSRF	服务间调用使用内部 API + INTERNAL_TOKEN 认证	✓

6.5.2 XSS 防护实战

前后端双重净化：

- 前端：DOMPurify 对 dangerouslySetInnerHTML 内容白名单过滤
- 后端：HtmlSanitizer.java 移除 <script>、onXXX 事件、javascript: 协议

6.5.3 文件上传安全

防护层	措施
Content-Type	仅允许 image/jpeg、image/png、image/webp
扩展名	白名单 .jpg/.jpeg/.png/.webp
大小限制	Spring 2MB + Service 层二次校验
路径遍历	过滤 ..、/、\ 字符
重命名	{userId}_{timestamp}.{ext} 格式

6.5.4 已知风险与缓解

风险	等级	缓解
CORS 开发配置	中	生产部署前更新为实际域名
文件本地存储	低	单实例 OK，生产建议迁移 OSS/S3
邮件服务依赖	低	不影响核心功能，控制台可输出验证码

七、前端设计过程

7.1 总体流程

- 第 1 步 — PRD 约定：根据业务逻辑，书写 PRD，约定好各页面的大致功能、页面路由以及前后端的接口约定
- 第 2 步 — Demo 验证：让 Agent 做一个 demo 页面，用于判断 Agent 是否正确理解目标美术风格。如果 Agent 写出的 demo 美术风格很差，则基于反馈纠正至满意为止
- 第 3 步 — 测试驱动：让 Agent 根据 PRD 的约定书写测试，采用测试驱动开发
- 第 4 步 — 分批开发：让 Agent 参考 demo，根据 PRD 的约定进行页面开发。如果页面太多，则尽量分组，防止上下文撑满（注：由于上下文压缩机制的不可控，Agent 可能会在压缩过程中遗漏重要信息，因此本人更倾向使用新 session 执行耦合度不高的各任务）

- **第 5 步 — 结果检验：** 检验方式包括第 3 步写的测试，还包括预先配置的 linter

7.2 美术风格探索

在确定最终方案前，经历了多轮模型与工具组合的迭代：

轮次	工具组合	输入形式	结果
第 1 轮	codex cli + gpt-5.4 + ui-ux-pro-max skill	文字提示词 / CC98 UI 图片	很丑，未达预期
第 2 轮	gemini-3.5flash + antigravity (Google 出品的 Agent GUI)	文字提示词	较 gpt 有改善，仍未达预期
第 3 轮	gemini-3.5flash + antigravity	UI 截图 + Playwright MCP (允许 Agent 访问网页并解析 DOM)	一次满足要求

关键体会： 文字描述的信息量不够高；将原始信息（图片、网页 DOM 等）作为输入，才能让 Agent 的产出风格收敛到期望结果。

7.3 页面开发与迭代

- **开发：** Agent 对分批页面进行开发
- **反馈：** 如果有细节需要改进，使用浏览器工具获取具体的元素名，并描述希望得到的效果，反馈给 Agent 进行优化
- **完成：** 循环迭代，直至页面开发完成